Internal Verification Behavior of LLMs

Tymur Mykhalievskyi

March 30, 2026

1 Introduction

Large Language Models (LLMs) are without a doubt one of the most significant advancements in the field of data science and artificial intelligence. They have shown remarkable capabilities in understanding and generating human-like text, making them invaluable tools for a wide range of applications. However, as these models become increasingly complex and powerful, we understand less about their internal workings and decision-making processes. This lack of transparency raises concerns about the reliability, robustness, and overall performance of LLMs.

The internal verification behaviors are natural to humans, as we often double-check our work to ensure accuracy and consistency. Seeing similar behaviors in the LLMs' thought process would, firstly, increase reliability, secondly, improve comprehension by making the trace of thoughts more thorough, and, last but not least, should positively influence the accuracy.

Hence, this proposal focuses on encouraging LLMs to use self-verification steps in the solutions. These can be seen as sanity checks or certain recalculations. These could include: sign parity checks during multiplication, unit consistencies or checks of total values such as sum of probabilities of all outcomes is 1.

2 Related Work

The current work on internal verification behaviors in LLMs mainly focuses on techniques that encourage the model to verify its own outputs via prompting strategies paired with multiple steps of sampling, verification and correction of some sort.

Furthermore, another approach that is frequently used, is to define an RL pipeline where the reward is affected by both correctness of the output and the quality of the verification process (Cai et al., 2026; Lee, Oh, Kim, Shin, & Tack, 2025; Zhang et al., 2025). While this approach is interesting and has shown promising results, it is not the focus of this research, as it requires a more complex setup and is not as widely used as the prompting strategies.

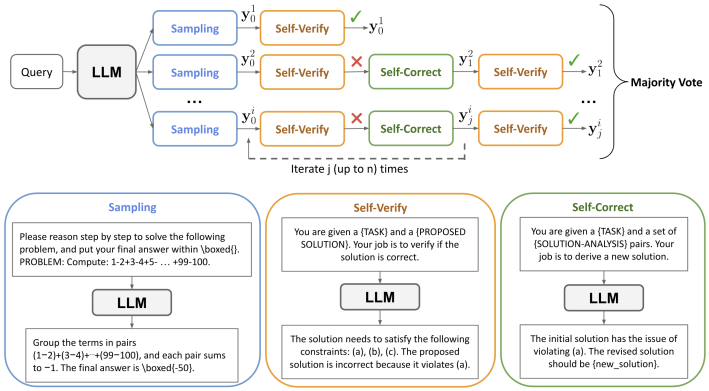

Figure 1: An illustration of the Self-Enhanced Test-Time Scaling (SETS) method (Chen et al., 2025), which involves "Self-Verify" and "Self-Correct" steps that recursively repeat until agreement. The approach uses multiple sampling and majority voting to determine the final answer.

Zhuang, Wang, Sun, and Lu (2026) proposed a simple yet effective method of prompting the model to find the solution, while independently generating facts about the problem. And afterwards, the model is prompted to verify the solution based on the generated facts. This method is particularly interesting as it encourages the model to utilize its internal knowledge and reasoning capabilities in a more comprehensive way, which is in line with the goals of this research. Similarly, we can design a method to generate domain and task related constraints and principles, and then prompt the model to verify the solution based on these constraints and principles. Andrade et al. (2026) show a similar idea, in a different domain of web navigation, where the model is prompted to independently generate an abstract solution and verify it with the proposed concrete solution.

Wu and Yao (2025) showed how prompting the model to firstly verify a solution before solving the task can lead to significant improvements in performance. This method is particularly interesting as it encourages the model to engage in a verification process before even attempting to solve the task, which can help to mitigate the agreement bias problem and lead to more accurate solutions.

Self-Enhanced Test-Time Scaling (SETS) is a method very close to the one proposed in this research (Chen et al., 2025). It involves "Self-Verify" and "Self-Correct" steps which recursively repeat until agreement. Furthermore, the approach uses multiple sampling and majority voting to determine the final answer. The main difference between SETS and our proposed method is how the "Self-Verify" step is implemented. See Figure 1 for an illustration of the method. SETS also utilizes ideas proposed by Wu and Yao (2025) by independently generating a solution and then verifying it.

3 Methodology

3.1 Proposed Method

The main idea behind the proposed method is to encourage LLMs to engage in a more comprehensive internal verification process that goes beyond just verifying the output of a task. The method involves prompting the model to not only verify the correctness of its output but also to consider domain-related constraints and principles that may not be directly related to the task at hand.

The main architecture is very similar to the one of SETS, the outer loop is a majority voting mechanism, where multiple samples are generated. However, the "Self-Verify" step will be split into two steps, the first one will be focused on generating the domain-related and/or task-related constraints and principles inspired by Zhuang et al. (2026), while the second step will be focused on verifying the solution based on these constraints and principles. This way, we can encourage the model to engage in a more comprehensive internal verification process that can potentially lead to improved performance. The "Self-Correct" step will be similar to the one in SETS, where the model is prompted to generate a new solution based on the feedback from the verification step. The process will repeat until agreement is reached among the generated samples or a certain limit of steps is reached.

Coming up with verification steps is not an easy task at all, as it requires not only being able to solve the task itself in the first place, but also to have a deeper fundamental understanding of the proposed problem. It is difficult to say, whether simply prompting the model to generate domain-related constraints will lead to the desired result. Hence, depending on the empirical results, the method might be further modified to include more specific instructions or examples to guide the model in generating relevant constraints and principles. For instance, a some rather simple examples of constraints are sign parity for multiplication or addition, or the total probability of all possible outcomes for a probability problem should be equal to 1. These constraints are not directly related to the task, but they are related to the domain and can be used to verify the correctness of the solution.

3.2 Competitor Methods

3.2.1 BoN and majority voting

This approach involves generating multiple responses to the same prompt and then using a voting mechanism to determine the most likely correct answer. By encouraging the model to produce multiple outputs, we can increase the chances of identifying the correct answer through consensus (Wang et al., 2023). This method has been shown to improve accuracy in various tasks, as it allows to look at the distribution of answers rather than just one particular solution. Essentially this approach is similar to the one proposed excluding the self-verification and self-correction steps, hence, it will be treated as a baseline for our experiments.

3.2.2 SETS

SETS is a method very close to the one proposed in this research, in fact, many architectural elements of the method are inspired by SETS. Hence, naturally, it will be treated as a competitor method in our experiments. The main difference between SETS and our proposed method is how the "Self-Verify" step is implemented. In SETS, the verification is only focused on the task and the output, while in our proposed method, we aim to explore a more comprehensive internal verification process that includes not only the task and output but also the domain related constraints not necessarily directly related to the task.

3.3 Datasets

The domain of mathematics naturally fits the goal of this research, as it often requires a deep understanding of the underlying concepts and principles, as well as the ability to apply them in various contexts. Furthermore, mathematical problems often allow for various verification steps that can be used to assess the correctness of the solution, making it an ideal domain for exploring internal verification behaviors in LLMs. Therefore, we will focus on mathematical datasets for our experiments. The main datasets that we will be using are GSM8K, MATH500, AIME 2024/2025, and AMC 2023. These datasets cover a wide range of mathematical problems, from basic arithmetic to more complex problems in algebra, geometry, and number theory. The difference in complexity should help us understand when such verification is most useful.

In addition, other domains might also be explored. While the improvements would not be expected, if the proposed method shows significant increases or decreases in performance, it would be interesting to investigate the underlying reasons for such behavior.

3.4 Evaluation Metrics

3.4.1 Accuracy

The accuracy fundamentally fits the goal of this research. As we hypothesize that the proposed verification method will lead to improved solutions, hence, increasing the accuracy.

3.4.2 Self-Verification Evaluation

A big role in this research is played by the self-verification process. A natural way to see how well the model is able to verify its own outputs is to evaluate the self-verification process itself. The evaluation can be done by computing the confusion matrix of the self-verification process, that is, how many times the model correctly identifies a correct solution as correct (true positives), how many times it incorrectly identifies a correct solution as incorrect (false negatives). Furthermore, metrics such as precision, recall, and F1-score can be computed to provide a more comprehensive evaluation of the self-verification process.

This evaluation not only gives insight into how much of an improvement in accuracy can be attributed to the self-verification process but also acts as a sanity check to ensure that the method is not hurting the performance of the model.

3.4.3 Test-Time Scalability

While improving accuracy is a primary goal, it is also important to consider the computational efficiency of the proposed method. The improvements in accuracy should not come at the cost of significantly increased computational resources or time.

3.5 Models

It is hard to depict the particular models that will be used in the experiments, as the research is still in its early stages. However, it is expected that the experiments will be conducted on a range of LLMs with open source weights, such as LLaMA or Qwen, with small to medium sizes of 1.5B to 7B parameters. The choice of models will depend on the computational resources available and the specific libraries that will be used for the experiments. It is expected that the lower size models will benefit more from the proposed method, as they have less capacity to store and utilize internal knowledge, hence, the verification process can help them to better leverage their internal knowledge and reasoning capabilities. On the other hand, the small models might not be able to generate the domain-related constraints as well as the larger ones.

4 Goals

4.1 Short-term Goals

The first and foremost goal of this research is to implement the proposed method and have clear evidence of models generating related constraints that could be used for verifying purposes.

Furthermore, the performance increase in combination to efficient test-time evaluation is clearly of the essence, otherwise, SETS and other competitor methods would be preferred.

4.2 Long-term Goals

The long-term goal of this research is to establish a deeper understanding of the internal verification behaviors and ideally reaching some generalization in terms of the applicability of the method across various domains and tasks.

References

Andrade, M., Cha, J., Ho, B., Srihari, V., Yadav, K., & Kira, Z. (2026). *Let's think in two steps: Mitigating agreement bias in mllms with self-grounded verification.* Retrieved from `https://arxiv.org/abs/2507.11662`

Cai, S., Qin, Y., Lin, H., Xu, Z., Li, G., Shi, Y., ... Sun, X. (2026). *Smart-snap: Proactive evidence seeking for self-verifying agents.* Retrieved from `https://arxiv.org/abs/2512.22322`

Chen, J., Ren, J., Chen, X., Yang, C., Sun, R., Yoon, J., & Arik, S. O. (2025). *Sets: Leveraging self-verification and self-correction for improved test-time scaling.* Retrieved from `https://arxiv.org/abs/2501.19306`

Lee, H., Oh, S., Kim, J., Shin, J., & Tack, J. (2025). *Revise: Learning to refine at test-time via intrinsic self-verification.* Retrieved from `https://arxiv.org/abs/2502.14565`

Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., ... Zhou, D. (2023). *Self-consistency improves chain of thought reasoning in language models.* Retrieved from `https://arxiv.org/abs/2203.11171`

Wu, S., & Yao, Q. (2025). *Asking llms to verify first is almost free lunch.* Retrieved from `https://arxiv.org/abs/2511.21734`

Zhang, F., Xu, J., Wang, C., Cui, C., Liu, Y., & An, B. (2025). *Incentivizing llms to self-verify their answers.* Retrieved from `https://arxiv.org/abs/2506.01369`

Zhuang, R., Wang, B., Sun, S., & Lu, K. (2026). Factored internal verification enhances large language model factuality. In T. Taniguchi et al. (Eds.), *Neural information processing* (pp. 258–272). Singapore: Springer Nature Singapore.